4. Cursor and selection.

T	e	s	t	‿	Ξ	ε	σ	κ	ε	π	ά	ζ	ω	‿	ر	ح	ي	م	‿	ر	ح	ي	م	‿	e	x	t	e	́	n	d	↵	i	t	ş	ţ	ş	ţ	
0054	0065	0073	0074	0020	039E	03B5	03C3	03BA	03B5	03C0	03AC	03B6	03C9	0020	0631	062D	064A	0645	0020	0631	062D	064A	0645	0020	0065	0078	0074	0065	0301	006E	0064	000A	0069	0074	015F	0162	015F	0162	0000

Points on the boundaries of different direction runs correspond to multiple visual positions.

Test Ξεσκεπάζω رحيم رحيم exténd itşţşţ

New LTR text will be inserted here.

And RTL text here.

Most UI toolkits draw both all the time.
To detect which one is more relevant use ether last entered text direction or selected keyboard layout.
Shortcut to manually switch primary cursor may be useful in case both of cursors don't fit to screen simultaneously.

Double cursor rendering variants:

w رحيم رحيم e w رحيم رحيم e

w رحيم رحيم e

Cursor in a multi-element cluster:

é ab 28...29

şt 356 2...3

tén st ? or ? tén st

Test Ξεσκεπάζω رحيم رحيم exténd itşţşţ

Selection of continuous range is not always continuous.

Cursor positions

Cursor positions calculation

```
int64_t leading_cluster = -1;
int64_t trailing_cluster = -1;
int64_t mid_cluster = -1;

if ((p_position == 0) && (data_size == 0)) {
    //no data - cursor at start

    ret.push_back(0.0f);

    return ret;
}
if (p_position > data_size) {
    //cursor after last char

    ret.push_back(get_cluster_leading_edge(data_size - 1));

    return ret;
}

for (int64_t i = 0; i < visual.size(); i++) {
    if (!visual[i].ignore_on_input) {
        if (((visual[i].start >= p_position) && (visual[i].end <= p_position)) || (visual[i].start == p_position)) {
            trailing_cluster = i;
        } else if (((visual[i].start >= p_position - 1) && (visual[i].end <= p_position - 1)) || (visual[i].end == p_position - 1)) {
            leading_cluster = i;
        } else if ((visual[i].start <= p_position) && (visual[i].end >= p_position)) {
            mid_cluster = i;
        }
    }
}
if ((leading_cluster != -1) && (trailing_cluster != -1)) {
    if ((p_primary_dir == TEXT_DIRECTION_RTL) && (visual[trailing_cluster].is_rtl)) {

        ret.push_back(get_cluster_leading_edge(leading_cluster));
        ret.push_back(get_cluster_trailing_edge(trailing_cluster));

    } else {

        ret.push_back(get_cluster_trailing_edge(trailing_cluster));
        ret.push_back(get_cluster_leading_edge(leading_cluster));

    }
} else {
    if (leading_cluster != -1) ret.push_back(get_cluster_leading_edge(leading_cluster));

    if (trailing_cluster != -1) ret.push_back(get_cluster_trailing_edge(trailing_cluster));
}

if (mid_cluster != -1) {
    float char_width = visual[mid_cluster].width / (visual[mid_cluster].end + 1 - visual[mid_cluster].start);
    int64_t pos_ofs = p_position - visual[mid_cluster].start;
    if (visual[mid_cluster].is_rtl) {

        ret.push_back(get_cluster_trailing_edge(mid_cluster) + pos_ofs * char_width);

    } else {

        ret.push_back(get_cluster_trailing_edge(mid_cluster) + pos_ofs * char_width);

    }
}
```

Generate script runs

Selection ranges calculation

Hit tests

```
float prev = 0.0f;
for (int64_t i = 0; i < visual.size(); i++) {
    float width = 0.0f;

    width += visual[i].width;

    if ((visual[i].start >= p_start) && (visual[i].end < p_end)) {

        ret.push_back(Rect2(prev, 0, width, descent + ascent));

    } else if ((visual[i].start < p_start) && (visual[i].end >= p_end)) {
        float char_width = visual[i].width / (visual[i].end + 1 - visual[i].start);
        int64_t pos_ofs_s = p_start - visual[i].start;
        int64_t pos_ofs_e = p_end - visual[i].start;

        ret.push_back(Rect2(prev + pos_ofs_s * char_width, 0, (pos_ofs_e - pos_ofs_s) * char_width, descent + ascent));

    } else if ((visual[i].start < p_start) && (visual[i].end >= p_start)) {
        float char_width = visual[i].width / (visual[i].end + 1 - visual[i].start);
        int64_t pos_ofs = p_start - visual[i].start;
        if (visual[i].is_rtl) {

            ret.push_back(Rect2(prev, 0, pos_ofs * char_width, descent + ascent));

        } else {

            ret.push_back(Rect2(prev + pos_ofs * char_width, 0, width - pos_ofs * char_width, descent + ascent));

        }
    } else if ((visual[i].start < p_end) && (visual[i].end >= p_end)) {
        float char_width = visual[i].width / (visual[i].end + 1 - visual[i].start);
        int64_t pos_ofs = p_end - visual[i].start;
        if (visual[i].is_rtl) {

            ret.push_back(Rect2(prev + pos_ofs * char_width, 0, width - pos_ofs * char_width, descent + ascent));

        } else {

            ret.push_back(Rect2(prev, 0, pos_ofs * char_width, descent + ascent));

        }
    }
    prev += width;
}

//merge intersectiong ranges
int64_t i = 0;
while (i < ret.size()) {
    int64_t j = i + 1;
    while (j < ret.size()) {
        if (ret[i].position.x + ret[i].size.x == ret[j].position.x) {
            ret[i].size.x += ret[j].size.x;
            ret.erase(ret.begin() + j);
            continue;
        }
        j++;
    }
    i++;
}
```

5. Structured text.

	Path:	Regular expression:	Source code:
Result of Unicode Bidirectional Algorithm: (incorrect)	`c:/אמת/שנים`	`[1,23][א-ב]`	`int תגובה*/;20 - B = A */`
Correct result:	`c:/שנים/אמת`	`[א-ן]{1,23}`	`int A = B - 20;/*תגובה*/`

BiDi is designed only for plain text and can not properly process text with higher order structure (paths, breadcrumbs, URLs, e-mail addresses, math formulas, dates, reg.exps., source code, etc)
Rendering structured text can be done either by tokenizing it and rendering each part individually or by injecting Unicode control chars.

PRI #185: Extension of UBA for improved display of URL/IRIs

Not implemented in the gdtl module.

CC 0